27 多執行緒演算法 • 獨立詳解

27-E01 Work、span 與處理器下界

推導

下界max(ceil(W/P),D)：P8為90，P16為45。強greedy上界(W−D)/P+D分別為696/8+24=111、696/16+24=67.5；離散單位時間故第二個可再寫T≤67。這些是區間，不能直接選其中一端當精確時間。

平均平行度720/24=30限制加速比，不限制P>30的邊際收益。指定DAG的第一階段須ceil(697/P)輪，之後鏈23輪，兩階段不能重疊；滿批執行第一階段可達此下界。P30為24+23=47，P31為23+23=46。

結論與檢核

P8時間在[90,111]，P16在[45,67]的greedy界內。指定DAG的第31顆確有收益；其加速比720/46仍小於30。

27-E02 遞迴矩陣的工作與相依

推導

八個乘法的work必相加，W(n)=8W(n/2)+Θ(n^2)，葉數8^(lg n)=n^3，得Θ(n^3)。span只沿一個child，再經Θ(lg n)的回收迴圈，所以D(n)=D(n/2)+Θ(lg n)。令n=2^L，各層span是Θ(L)、Θ(L−1)、…，和Θ(L^2)。

8不是只在根出現的乘法常數，會使每層呼叫數再乘8；改成1會改變整棵樹。C11的ae與bg若都以非原子+=更新，便共同讀改寫C11每一格；C12、C21、C22各自也有相同衝突。

結論與檢核

W=Θ(n^3)、D=Θ($\lg^2 n$)、平均平行度Θ($n^3/\lg^2 n$)。分到C與獨立T、等全部完成後逐格加回，是解除此競爭的一種完整做法。

27-E03 Parallel merge 的最壞 span

推導

n1≥n2、N=n1+n2。median兩側第一run至多floor(n1/2)，即使第二run全落同一側，child≤n1/2+n2=(N+n2)/2≤3N/4。二分另一run最壞O(1+lg(n2+1))≤O(lg(N+1))，遞迴深O(lg N)，逐層log大小以常數下降，總和O($\lg^2$(N+1))。

取等長2冪的偶數run與奇數run；左child保留前半偶數、前半奇數，依舊同型。每層搜尋的另一run仍有與當前總長同階的元素，中間切點需Θ(log size)次比較，累加給Ω($\lg^2 N$)。另一run空時，每次搜尋是常數，只剩較長run的median遞迴樹，其span Θ(lg(N+1))。

27 多執行緒演算法 • 獨立詳解 （續）

結論與檢核

最壞$\Theta(\lg^2 N)$，不是每個輸入都同樣昂貴。只知道$O(\lg N)$層，不能把每層二分搜尋的成本忽略。

27-E04 九個 Fibonacci strand 的排程

推導

根有pre3、cont3、join3；左P-FIB2有pre2、cont2、join2、F1a、F0；根右child是另一個F1b。依賴為pre3→{pre2,cont3}，cont3→F1b，pre2→{F1a,cont2}，cont2→F0，{F1a,F0}→join2，{join2,F1b}→join3。

時間步	P=3所執行strand
1	pre3
2	pre2、cont3
3	F1a、cont2、F1b
4	F0
5	join2
6	join3

W=9，最長路pre3、pre2、cont2、F0、join2、join3長6，故D=6、平均平行度3/2。表中每次執行全部ready者，為greedy，且達D下界，因此時間6最短。

令$Z_n=W_n+3$，則$Z_n=Z(n-1)+Z(n-2)$，$Z_0=Z_1=4$，所以$Z_n=4F(n+1)$、$W_n=4F(n+1)-3$。$D_0=D_1=1$，$D_2=4$；代入$D(n)=2+\max(D(n-1),1+D(n-2))$，n3得6，之後歸納$D_n=2n$。

結論與檢核

P-FIB3為W9、D6，P3最短6步。n4得W17、D8，與相同strand模型一致。

27-E05 遺失更新與兩種修復

推導

合法次序：T1讀0到r1；T2讀0到r2；T1令r1=1並寫x=1；T2令r2=1並寫x=1。兩task內部順序均保留，最後仍1。結尾sync只等兩者完成，沒有回溯更改已遺失的更新。

n次真正原子加對同一格雖可保證和，但在這個單格序列化模型有n次依賴，span $\Theta(n)$。分散法讓task i只寫局部v[i]，再以二分reduction合併；每個父只加兩個已完成child結果，work $\Theta(n)$、span $\Theta(\lg(n+1))$。需加法可結合及獨立儲存；浮點重排的捨入結果另判。

27 多執行緒演算法 • 獨立詳解 （續）

結論與檢核

sync不會使先前load/add/store成原子。原子化與reduction都可修正答案，但依賴結構及span不同。

27-E06 矩形矩陣的 reduction

推導

對每個(i,j)獨立計q長內積，nested parallel for遍歷i∈[0,p)、j∈[0,r)。內積DOT(klo,khi)在空區間回0、單元素回A[i,k]B[k,j]；其他情況二分k區間，spawn左、call右、sync後回left+right。每個輸出只由其一個DOT根寫入Cij。

每個DOT有q乘法、q−1加法，work Θ(q)、span Θ(1+lg q)；外層發出成本Θ(pr)，不超過Θ(pqr)。總span Θ(1+lg p+lg r+lg q)，p=q=r=1時仍Θ(1)，不能把lg1=0當成零工作。

若各(i,j)的reduction樹完全同時展開，最多Θ(pqr)個活躍task／局部中間結果；輸入pq+qr與輸出pr個格另外計。深度優先或有限P可減少活躍workspace，不能據此對所有排程承諾相同峰值。配置採正文未初始化空間模型，若串行初始化則需另計。

結論與檢核

work Θ(pqr)，span Θ(1+lg p+lg r+lg q)，無同格並行累加。此界在任一維等於1時仍有意義。

27-E07 較長 run 交換會破壞哪種平手規則

推導

只比key時，先交換使第一run=[1b,2c]，第二=[1a]。upper median是2c，lower_bound在[1a]返回末端，放O2=2c。左child合併[1b]與[1a]，median為1b，而lower_bound以key比較發現1a≥1，切點在最前，放O0=1b；剩1a寫O1。結果[1b,1a,2c]不穩定。

以(key,原始位置)比較，1a<1b<2c；第二步lower_bound(1b)越過1a，放O1=1b，剩1a寫O0。一般而言descriptor交換不改record的固定總序，所有等key者的原始位置仍決定輸出順序，所以歸納合併保持穩定。

結論與檢核

key-only規則雖仍按key排序，卻不穩定；全域唯一身分的總序修復為[1a,1b,2c]。兩run的暫時角色不能取代原始先後。

27 多執行緒演算法 • 獨立詳解 （續）

27-E08 輸出 offset 與別名

推導

m=2、pivot6c；lower_bound另一段給j=6，因3f<6c而6g>6c。q=3+(2−0)+(6−4)=7。左輸出[3,7)長4，pivot在7，右輸出[8,11)長3。

最終O[3..11)為[1a,2e,3f,4b,6c,6g,8h,9d]。其他O格不應改寫；o只是加到每個相對秩，並未改輸入切點。

若O=T、o=0，根會先把T4從2e覆寫成6c；但左child仍須讀原T[4..6)的2e、3f。該值已失去，甚至此run不再排序，因此不能用這個程序直接原地合併。

結論與檢核

q7，左右區間[3,7)、[8,11)。輸出與未讀輸入不重疊是正確性前提，不能只由輸出區間彼此不重疊就推得全部安全。

27-E09 從 merge 遷移到一般平衡樹計費

推導

對大小m節點，把c lg(m+1)費平均收自其m個record。固定record沿ancestor鏈收費到自己成pivot；從最後向上r層，大小至少ρ^(−r)。因g(u)=lg(u+1)/u遞減，該層收費≤cρ^r lg(ρ^(−r)+1)≤cρ^r[1+r lg(1/ρ)]。

Σρ^r與Σrρ^r都收斂，因此每record總費O(1)，常數依固定ρ而定，總O(N)。若每節點最多固定個數child且每非空呼叫拿走一record，總空呼叫O(N)；merge為二元，恰N+1個空呼叫。若不限制child個數，空呼叫數本身可能任意大，則題設需補這個控制成本前提。

若每次只拿走一record、餘N−1全進一個child，比例1−1/N隨N趨1。每層成本取lg(m+1)，總Σm=1..N lg(m+1)=lg((N+1)!)=Θ(N lg N)，不能用固定ρ的收斂常數。

結論與檢核

固定縮小比例、固定最大child數、每次唯一拿走一record，使一般總work O(N)。原題若未明列固定child數，必須補足後才能連空呼叫成本一起保證；這不是可由記帳自動消失的費用。

27-E10 最壞 sort span 需要同一輸入族

推導

每一層單獨可有昂貴merge，不表示有同一個輸入同時讓全部層昂貴；下界需要相容構造。n16的4-bit反轉順序為[0,8,4,12,2,10,6,14,1,9,5,13,3,11,7,15]。根兩半排序後分別[0,2,4,6,8,10,12,14]與[1,3,5,7,9,11,13,15]。

27 多執行緒演算法 • 獨立詳解 （續）

初始前半全為偶數，除2後其相對秩是n8的反轉順序；後半減1除2也相同。遞迴每層均有此同型性，所以各層merge輸入都為近等長交錯run，沿路需Ω(log²(size))。parent的merge必在兩child完成後，故一條關鍵路徑可串接最長child與本層最長merge路徑。

結論與檢核

Σr=1..L r²=Θ(L³)，配一般上界得最壞Θ(lg³n)。n16各尺度log大小4、3、2、1的平方和30是此級數的數值示意，不是未定義常數模型下的精確執行時間。

27-E11 Work 與 span 的優劣翻轉

推導

保證B快的一個充分條件是B的上界1536/P+32小於A的work下界2048/P，即P<16，所以整數1≤P≤15均可。保證A快的一個充分條件是A上界2048/P+8小於B的span下界32，即P>256/3，所以P≥86均可。

在16≤P≤85，這兩個充分條件都未成立，不能由其失敗反推兩者一樣快，也不能把兩個上界相減便當成實際時間差。可再用較強greedy界收緊區間，或取得DAG／排程資訊；本題指定的鬆界未作出判定。

結論與檢核

上述範圍為足夠保證，沒有宣稱是最佳可判定範圍。Work較少但span較大的算法，在不同P上可能翻轉優劣。

27-E12 依序 spawn 不等於平行 for

推導

令r=ceil(n/g)。普通for有r個依序發出的spawn，形成Θ(r)parent鏈；一個批次本身至多g個單位工作、最壞Θ(g)。整體span上界O(r+g)，下界Ω(max(r,g))亦為Ω(r+g)，故Θ(n/g+g)，work Θ(n+r)=Θ(n)。

由n/g+g≥2√n，取整數g在√n附近達Θ(√n)span與Θ(√n)平行度。平衡發出樹則只需Θ(lg r)控制深度，span Θ(lg r+g)。g=1在此抽象常數模型給Θ(lg n)，但實際大量細task的常數開銷可使固定較粗粒度更好。

結論與檢核

發出方式本身改變span，不能把普通for上的spawn當成PAR-FOR。兩者work皆線性，並不代表關鍵路徑相同。

27 多執行緒演算法 • 獨立詳解 （續）

27-E13 少暫存的矩陣乘法

推導

以下輸入階數先補為2冪，C與只讀A、B獨立；外層平行清C=0，再呼叫ACC。

```
ACC(C,A,B):
    n = A.rows
    if n == 1: C[0,0] += A[0,0]*B[0,0]; return
    view A as [[a,b],[c,d]], B as [[e,f],[g,h]]
    view C into C11,C12,C21,C22
    spawn ACC(C11,a,e); spawn ACC(C12,a,f)
    spawn ACC(C21,c,e); ACC(C22,c,f)
    sync
    spawn ACC(C11,b,g); spawn ACC(C12,b,h)
    spawn ACC(C21,d,g); ACC(C22,d,h)
    sync
```

每階段四個child只更新四個不同C區塊，無衝突；兩階段之間sync，確保同一C區塊的第一個乘積全部加完才加第二個。對階數歸納，兩階段合計各自正確的區塊乘積，結果為原C+AB。

W(n)=8W(n/2)+Θ(1)=Θ(n^3)，清零Θ(n^2)不主導。D(n)=2D(n/2)+Θ(1)=Θ(n)，清零Θ(lg n)也不主導。無額外n×n數值暫存；呼叫／task描述符最多O(n^2)，單一路徑stack O(lg n)，A、B、C三矩陣另計Θ(n^2)格。

結論與檢核

平均平行度Θ(n^2)，低於八乘法同時展開的Θ(n^3/$\lg^2 n$)，但避免其字面實作的Θ(n^3)峰值工作區。少空間是用兩階段相依換來，不能仍保留原$\log^2$ span。

27-E14 隨機 DAG 的兩種平均加速比

推導

第一種可用10條各長10的獨立鏈，W100、D10，在P10用10步；第二種是一條長100的鏈，P10仍100步。兩種等機率時，E[W/T10]=(10+1)/2=11/2；E[W]/E[T10]=100/55=20/11。

前者是逐次speedup的算術平均；後者是固定一批實驗累積工作與累積時間的比。例如各跑一次，總W200、總T110，比例20/11。兩者是不同統計量，不能互換。

對相同亂數分布產生的每一個DAG，逐點有Tp≥W/P、Tp≥D、greedy Tp≤(W−D)/P+D。取期望並用線性性得到

$$\max\{E[W]/P, E[D]\} \le E[T_P] \le E[W]/P + (1 - 1/P)E[D].$$

27 多執行緒演算法 • 獨立詳解 （續）

結論與檢核

不需要W、D獨立；此處用的只是逐次不等式及線性性。若討論一批工作總量的加速比，採E[W]/E[Tp]，並以E[W]/E[D]提供相應上界。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

27 多執行緒演算法 · 獨立詳解 （續）

原創驗算程式附錄: parallel_depth_checks.py

```
"""Executable finite models accompanying the general proofs; not a timing benchmark."""
from itertools import combinations, combinations_with_replacement, product
from functools import lru_cache
from fractions import Fraction
from random import Random

SAMPLE=[(1,0),(4,1),(6,2),(9,3),(2,4),(3,5),(6,6),(8,7)]

def fib_dag(n,prefix=''):
    if n<2:
        name=prefix+'b';return {name:f'F{n}'},[],name,name
    L,le,ls,lt=fib_dag(n-1,prefix+'L');R,re,rs,rt=fib_dag(n-2,prefix+'R')
    p,c,j=(prefix+s for s in ('p','c','j'))
    return {**L,**R,p:f'pre{n}',c:f'cont{n}',j:f'join{n}'},le+re+[(p,ls),(p,c),(c,rs),(lt,j),
(rt,j)],p,j

def span(nodes,edges):
    pred={v:set() for v in nodes}
    for u,v in edges:pred[v].add(u)
    @lru_cache(None)
    def depth(v):return 1+max((depth(u) for u in pred[v]),default=0)
    return max((depth(v) for v in nodes),default=0)

def greedy(nodes,edges,P,reverse=False):
    pred={v:set() for v in nodes}
    for u,v in edges:pred[v].add(u)
    done=set();steps=[]
    while len(done)<len(nodes):
        ready=sorted((v for v in nodes if v not in done and pred[v]<=done),reverse=reverse)
        assert ready
        batch=ready[:P];steps.append(batch);done.update(batch)
    return steps

def merge(T,l1,r1,l2,r2,O,o,trace=None,key=lambda x:x,right_first=False):
    """Same half-open ranges and separate-output contract as the text."""
    assert T is not O
    if trace is None:trace=[]
    if r1-l1<r2-l2:l1,r1,l2,r2=l2,r2,l1,r1
    n1,n2=r1-l1,r2-l2
    if not n1:
        trace.append({'empty':True});return trace
    m=(l1+r1)//2;x=T[m];lo,hi=l2,r2;search=[]
    while lo<hi:
        mid=(lo+hi)//2;search.append((lo,hi,mid,T[mid]))
        if key(T[mid])<key(x):lo=mid+1
        else:hi=mid
    j=lo;q=o+(m-l1)+(j-l2)
    left=m-l1+j-l2;right=r1-m-1+r2-j
    assert left+right==n1+n2-1 and 4*max(left,right)<=3*(n1+n2)
```

Lin Cing https://github.com/xinhi6716/computer-science-self-study

27 多執行緒演算法 · 獨立詳解 （續）

```
    O[q]=x

trace.append(dict(empty=False,l1=l1,r1=r1,l2=l2,r2=r2,m=m,j=j,q=q,pivot=x,N=n1+n2,search=sear
ch))
    args=[(l1,m,l2,j,o),(m+1,r1,j,r2,q+1)]
    for a,b,c,d,out in args[::-1] if right_first else args:
        merge(T,a,b,c,d,O,out,trace,key,right_first)
    return trace

def sort(values,right_first=False):
    original=[(key,i) for i,key in enumerate(values)];X=original.copy();Y=original.copy()
    def go(src,dst,l,r):
        assert src[l:r]==original[l:r] and dst[l:r]==original[l:r]
        if r==l:return
        if r-l==1:dst[l]=src[l];return
        m=(l+r)//2;calls=[(l,m),(m,r)]
        for a,b in calls[::-1] if right_first else calls:go(dst,src,a,b)
        merge(src,l,m,m,r,dst,l,right_first=right_first)
    go(X,Y,0,len(X));return Y

def dot(a,b):
    assert len(a)==len(b)
    if not a:return 0,0,0
    if len(a)==1:return a[0]*b[0],1,1
    m=len(a)//2;x,w,d=dot(a[:m],b[:m]);y,W,D=dot(a[m:],b[m:])
    return x+y,w+W+1,max(d,D)+1

def multiply(A,B):
    n=len(A)
    if n==1:return [[A[0][0]*B[0][0]]]
    h=n//2
    def part(M,i,j):return [row[j:j+h] for row in M[i:i+h]]
    C=[[0]*n for _ in range(n)];T=[[0]*n for _ in range(n)]
    for i,j in product((0,h),repeat=2):
        first=multiply(part(A,i,0),part(B,0,j));second=multiply(part(A,i,h),part(B,h,j))
        for u,v in product(range(h),repeat=2):C[i+u][j+v]=first[u][v];T[i+u][j+v]=second[u]
[v]
    return [[C[i][j]+T[i][j] for j in range(n)] for i in range(n)]

def accumulate(C,A,B):
    n=len(A)
    if n==1:C[0][0]+=A[0][0]*B[0][0];return
    h=n//2
    def part(M,i,j):return [row[j:j+h] for row in M[i:i+h]]
    for k in (0,h):
        for i,j in product((0,h),repeat=2):
            block=part(C,i,j);accumulate(block,part(A,i,k),part(B,k,j))
            for u,v in product(range(h),repeat=2):C[i+u][j+v]=block[u][v]

def bit_reverse(n):
```

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

27 多執行緒演算法 · 獨立詳解 （續）

```
    k=n.bit_length()-1;assert n==1<<k
    return [int(f'{i:0{k}b}'[::-1],2) if k else 0 for i in range(n)]

def interleaved_worst(n):
    def go(keys):
        if len(keys)<2:return keys
        first=keys[::2] if len(keys)%2==0 else keys[1::2]
        second=keys[1::2] if len(keys)%2==0 else keys[::2]
        return go(first)+go(second)
    return go(list(range(n)))

def check():
    result={};fib=[0,1]
    for _ in range(13):fib.append(sum(fib[-2:]))
    for n in range(12):
        nodes,edges,_,_=fib_dag(n);W=len(nodes);D=span(nodes,edges)
        assert W==4*fib[n+1]-3 and D==(1 if n<2 else 2*n)
    N,E,_,_=fib_dag(3);assert len(greedy(N,E,3))==6
    result['fibonacci_sizes']=12
    dag_cases=0
    for n in range(1,6):
        possible=list(combinations(range(n),2))
        for mask in range(1<<len(possible)):
            edges=[e for i,e in enumerate(possible) if mask>>i&1];D=span(range(n),edges)
            for P in range(1,n+1):
                for reverse in (False,True):
                    t=len(greedy(range(n),edges,P,reverse))
                    assert max((n+P-1)//P,D)<=t<=Fraction(n-D,P)+D
                    dag_cases+=1
    result['greedy_small_dag_schedules']=dag_cases
    assert (697+29)//30+23==47 and (697+30)//31+23==46
    cases=0
    runs=[a for n in range(6) for a in combinations_with_replacement(range(3),n)]
    for a,b in product(runs,repeat=2):
        T=[(v,i) for i,v in enumerate(a+b)];n=len(T)
        for rev in (False,True):
            O=[None]*(n+6);trace=merge(T,0,len(a),len(a),n,O,3,right_first=rev)
            assert O[:3]==[None]*3 and O[n+3:]==[None]*3 and O[3:n+3]==sorted(T)
            writes=[s['q'] for s in trace if not s['empty']]
            assert len(trace)==2*n+1 and sorted(writes)==list(range(3,n+3));cases+=1
    result['stable_merge_range_and_serialization_cases']=cases
    O=[None]*8;trace=merge(SAMPLE,0,4,4,8,O,0)
    assert [s['q'] for s in trace if not s['empty']]==[4,3,2,1,0,6,5,7]
    assert trace[0]['j']==6 and trace[0]['m']==2 and O==sorted(SAMPLE)
    T=[(1,0),(1,1),(2,2)];O=[None]*3;merge(T,0,1,1,3,O,0,key=lambda v:v[0]);assert O==[(1,1),
(1,0),(2,2)]
    arrays=0
    for n in range(9):
```

27 多執行緒演算法 • 獨立詳解 （續）

```
        for values in product(range(3),repeat=n):
            for rev in (False,True):assert sort(values,rev)==sorted((v,i) for i,v in
enumerate(values))
            arrays+=1
    result['two_buffer_stable_sort_arrays_both_orders']=arrays
    for k in range(1,9):
        n=1<<k;v=bit_reverse(n)
        assert sorted(v[:n//2])==list(range(0,n,2)) and sorted(v[n//2:])==list(range(1,n,2))
    result['bit_reversal_dyadic_sizes']=8
    for n in range(1,130):
        v=interleaved_worst(n)
        assert sort(v)==[(i,v.index(i)) for i in range(n)]
        def inspect(a):
            if len(a)<2:return
            h=len(a)//2;left=sorted(a[:h]);right=sorted(a[h:]);order=sorted((x,0) for x in
left)+sorted((x,1) for x in right)
            origins=[side for _,side in sorted(order)]
            assert all(origins[i]!=origins[i+1] for i in range(len(origins)-1))
            inspect(a[:h]);inspect(a[h:])
        inspect(v)
    result['all_levels_interleaved_nondyadic_sizes']=129
    outcomes=set();interleavings=0
    for slots in combinations(range(6),3):
        order=[0 if t in slots else 1 for t in range(6)];pc=[0,0];reg=[0,0];x=0
        for who in order:
            if pc[who]==0:reg[who]=x
            elif pc[who]==1:reg[who]+=1
            else:x=reg[who]
            pc[who]+=1
        outcomes.add(x);interleavings+=1
    assert outcomes=={1,2};result['two_task_load_add_store_interleavings']=interleavings
    assert dot([3,-1,2,4],[2,5,-3,1])==(-1,7,3)
    rng=Random(2704);matrices=0
    for n in (1,2,4,8):
        for _ in range(20):
            A=[[rng.randrange(-5,6) for _ in range(n)] for _ in range(n)];B=
[[rng.randrange(-5,6) for _ in range(n)] for _ in range(n)]
            AB=[[sum(A[i][k]*B[k][j] for k in range(n)) for j in range(n)] for i in range(n)]
            assert multiply(A,B)==AB
            C=[[rng.randrange(-5,6) for _ in range(n)] for _ in range(n)];old=[r[:] for r in
C];accumulate(C,A,B)
            assert C==[[old[i][j]+AB[i][j] for j in range(n)] for i in range(n)];matrices+=1
    result['eight_product_and_two_phase_matrix_pairs']=matrices
    assert Fraction(10+1,2)==Fraction(11,2) and Fraction(100,55)==Fraction(20,11)
    return result

if __name__=='__main__':
    import json
    print(json.dumps(check(),indent=2))
```

Lin Cing https://github.com/xinhi6716/computer-science-self-study

27 多執行緒演算法 · 獨立詳解 （續）

原創驗算程式附錄: work_span_bounds.py

```
"""Compute work, span, parallelism, and processor lower bounds."""

def bounds(work, span, processors):
    assert work >= span > 0 and processors > 0
    return {
        "work": work,
        "span": span,
        "parallelism": work / span,
        "processor_lower_bound": max(work / processors, span),
    }


assert bounds(720, 24, 8)["processor_lower_bound"] == 90
assert bounds(720, 24, 16)["processor_lower_bound"] == 45
assert bounds(1000, 50, 32)["processor_lower_bound"] == 50
print(bounds(720, 24, 16))
```

參考資料與署名

Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein. Introduction to Algorithms, 3rd Edition.

Lin Cing 原創教學整理。原創內容以 CC BY 4.0 授權；參考書籍不隨教材散布。

https://github.com/xinhi6716/computer-science-self-study